

Beatbox: Written Evaluation

Danielle Whisnant (dwhisnan), Domingo Viesca (dviesca), Javier Niño-Sears (jninosea)

GitHub: <https://github.com/Domingo-v/beatbox-1470>

Introduction

In the last few decades, streaming platforms have revolutionized access to music, with more music being easily accessible now than at any other time in history. Complex ML algorithms underlie all of these platforms, opening up new pathways for artists to achieve increased discovery and revenue. One such algorithm that Spotify, for example, relies on is its genre classification and tagging system. While Spotify employees are mostly responsible for monitoring the creation of new genres as listening habits and musical categories evolve over time, Spotify has an in-house algorithm that classifies songs into these genres. We set out to replicate this genre classification system, to gain a deeper understanding of how algorithms can be used to shape musical taste and discovery.

This was implemented in a previous paper, “Music Genre Classification using Machine Learning Techniques”, wherein the authors used several strategies for genre classification. Primarily, they fine-tuned a pre-trained CNN model to analyze spectrographic data (which yielded the highest accuracy). Separately, they analyzed feature data using SVMs, random forest, gradient boosting (XGB), and logistic regression.

Methodology

The model paper suggested using a fine-tuned, pre-trained CNN to analyze spectrographic data and separate machine learning algorithms (SVMs, XGB, random forest) to analyze feature data. We combined these techniques into one pipeline by continuing to use a 2D CNN for the spectrographic data, using an MLP with Dropout for our feature data, and concatenating the two before using a final dense output layer with softmax activation. For comparison, we also used solely the spectrographic data with the 2D CNN to see if this would yield better results (as in the model paper). We used trial-and-error to tune our hyperparameters, including: the learning rate, embedding size, and regularization strength. We also used general observation to select the number of epochs for training according to local and online computational resources (Google colab).

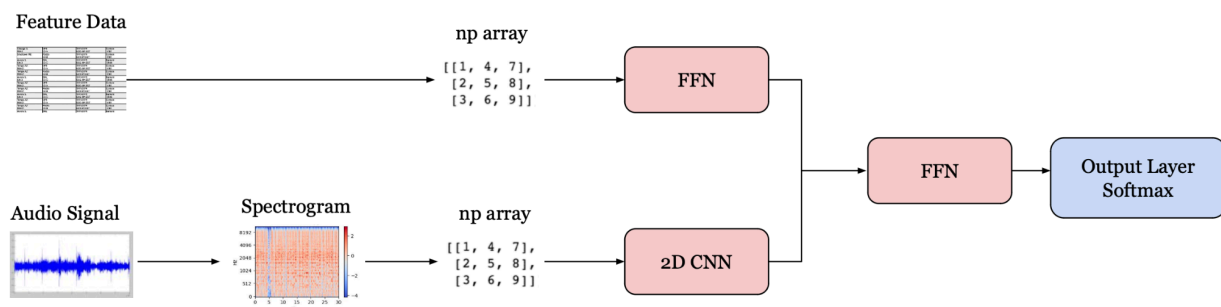


Figure 1. Algorithm Flowchart

We obtained our data through the Free Music Archive (FMA), one of the largest current audio-related open-source projects, and used the smallest batch of data in the database, which had 8,000 .mp3 files—1,000 files each per genre. This dataset separately included tabular data that described the features of *a subset* of the audio files. Sample features included danceability, tempo, speechiness, and acousticness.

Data and Preprocessing:

Initially, we trialed our model and data processing on a smaller dataset of .wav files. We utilized this dataset to troubleshoot and iterate on our preprocessing script and model pipeline quickly before moving on to the much larger dataset. We considered .wav files to be optimal from their lossless and uncompressed nature from which the CNN pipeline could pick up more granular patterns. .Wav files are less common among large open source datasets, so when modifying the codebase to work with the FMA dataset which is principally .mp3 files, we added functionality for .mp3 files which are more common and less memory intensive. To read in and manipulate the audio files we made use of the *Librosa* library and got familiar with FMA-built *Utils*.

For the transformation-preprocess pipeline: once the files are loaded in, the audio signals are converted to a mel spectrogram image. To sharpen spectral detail, a filter is passed through the image to emphasize and sharpen high frequencies that might initially be hard to pick up on. Initially, we had clipped out silent sections of a given song, but after trials, we discovered that pauses and silence were an essential feature to keep for more accurate classification. Each spectrogram is also clipped or padded to fit within the input shape before being transformed into a more lightweight numpy format. Once augmented, each spectrogram image is mapped from spectrogram pixel value (power) to decibels in log scale on a numpy array. Once in array format, the decibel values are normalized to smooth out activation between songs that might have been loaded in at different volumes. The process is repeated for each audioclip in the dataset and exported to .npy files of shape (96, 1293) with song_id as the filename such that the preprocess function only has to be run once during hyperparameter tuning.

A second process was also implemented, but only applies to the pipeline that includes tabular features. Audio feature data is loaded in from a separate CSV file. Features are persistent across the dataset; thus, we elected to use one-hot encoding for their representation. The table and audio file correspondences are mapped using the song_id and loaded in order to a numpy array for concatenation after vectorization further down the model pipeline.

After training the models, we measured accuracy and AUC to understand performance of the model. We also generated LIME explanations for randomly-selected spectrograms for better interpretability.

Results

After tuning our hyperparameters and architecture, we optimized the 2D CNN's performance at an accuracy of 0.49 and AUC of 0.68. This was after running the model for 15 epochs with a learning rate of 0.00, a lambda of 0.02 for regularization, and early stopping with patience of 10.

The combined 2D CNN and tabular data MLP model was optimized to an accuracy of 0.51 and an AUC of 0.70. These results came after 30 epochs, with a learning rate of 0.001, a lambda of 0.02 for regularization, and early stopping with patience of 10. Due to mismatched tabular and spectrographic data, this model only processed 1,035 audio files and their feature data, instead of the full 8,000 audio files provided in the FMA dataset.

Model	Accuracy	AUC
2D CNN	0.49	0.68
2D CNN + Tabular Data MLP	0.51	0.70
Paper's Fine-tuned, Pre-trained 2D CNN	0.64	0.89

Figure 2. Accuracy and AUC Results

We also generated LIME explanations for the spectrograms fed into the 2D CNN. These were randomly selected from the sample set and comprised both correct and incorrect classifications. Two LIME explanations for classification into the rock genre (one correct and one incorrect) are shown below. In both, it is apparent that our model highlighted the region in (frequency = ~40, time = 800-1000) as the most important feature for classifying an audio file as "rock." The region in (frequency = 50, time = 200-400) was also highlighted as relevant in this classification.

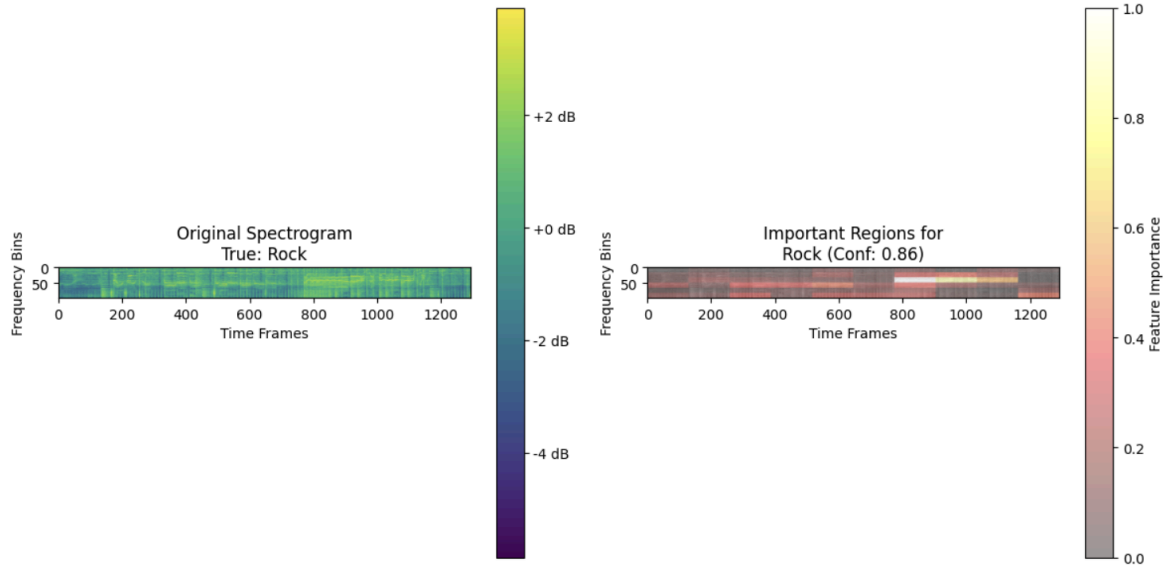


Figure 3. Correct Classification in Rock Genre with Confidence = 0.86

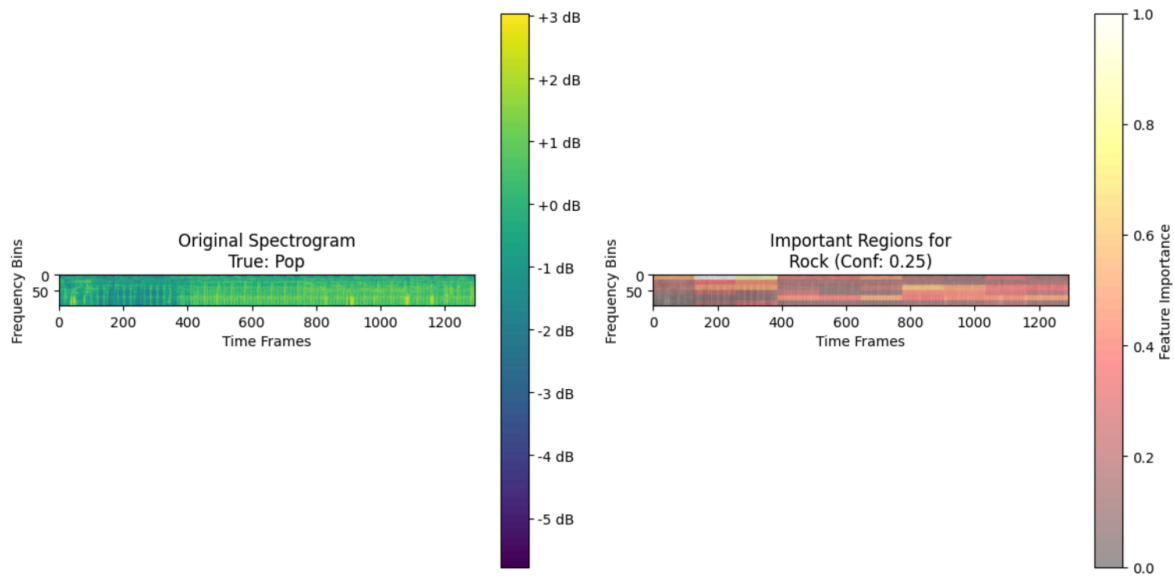


Figure 4. Incorrect Classification in Rock Genre with Confidence = 0.25

Challenges

A significant challenge in the early stages of this project was in preprocessing the highly complicated dataset. The dataset proved extremely hard to work with due to its sheer scale. Both the table of song metadata, which included items such as unique identifiers and genre (the ground truth label), and the features table were very large, formatted in more than three dimensions, and had a propensity to crash local and online resources which added a layer of technical tinkering to the project. Furthermore, the table of extra features did not have

information for every song in the dataset we used, which was not noted in the data spec, and limited the extent of data our CNN + Tabular-features pipeline would work with down the road. Using the larger 100+ gigabyte dataset to capture a larger subset of songs with features proved feasible with our resources, but would be interesting and easily implementable as a follow-up to this project.

For the missing data, we decided to drop a sizable number of spectrograms during preprocessing. We needed to include a matching/concateration process between the spectrograms and the tabular data, because the tabular data needed to be linked to corresponding spectrograms. As such, we could only use the spectrograms that had matches in the tabular data, and adding in an empty representation only caused slowdowns in the training process. Once already in the training process, we discovered there were only 1,035 spectrograms that matched with feature data, down from the original 8,000 spectrograms we had created. This setback prompted us to hyper-optimize for efficiency. We had to work around this mismatch in the dataset and with it, the diminished performance of our combined model. If we had the capacity to find all of the 8,000 tabular data records that matched with our 8,000 spectrograms, our model would have likely recorded much higher accuracy on the combined model than we were able to with our limited data.

Reflection

We were very pleased with how our project turned out. Without context, our accuracy would seem low, but when compared to the accuracy of our model paper, we achieved a relatively close accuracy with much fewer resources, compute, and data. There were some unexpected obstacles along the way, that given what we know now, we may have approached differently. For example, if we had realized the extent of dropped spectrograms in the preprocessing phase, we would have continued looking to find an expanded tabular dataset so that our combined model could have more robust data to train on. We were also constrained by storage considerations on our personal devices. We chose to use Free Music Archive's small dataset, which was 7.2 gigabytes. The larger sizes of data would have been 22, 93, or 879 gigabytes. Any of these datasets would have likely achieved higher levels of accuracy, but our personal devices could not reasonably handle storing, processing, or training on such large datasets. With expanded or cloud-based storage systems, and stronger hardware that could handle datasets of that size, we could have increased accuracy even more.

Our biggest takeaway from this project is that it is very difficult to classify music into genres. The very concept of a "genre" is human-created. Spotify employs many musicologists, whose entire job it is to define genres, based on developments in musicological research, trends in user listening habits, and new types of music uploaded to the platform. While the classification of songs into genres at Spotify is algorithmic, the definition and delineation of those genres is inherently human. As such, it is difficult for classification algorithms to keep up with the

rapidly-evolving definitions of music genres. The eight genres in our dataset ranged in specificity, with genres like “Rock” and “Hip Hop” being easier to predict than genres like “Experimental” or “International”. As expected, the latter two genres had the worst classification accuracy in our runs by a wide margin. Genres like [sambass](#), [maskandi](#), and [waiata maori](#) have practically nothing in common, yet would all be grouped under “International”, which a model would not be able to figure out from waveforms and tabular data alone. As Spotify’s international usership [broadens year over year](#), seen in Figure 5 below, it is vital for Spotify’s musicology department to become more international as well, and to develop more nuanced genre breakdowns than existed in our data.

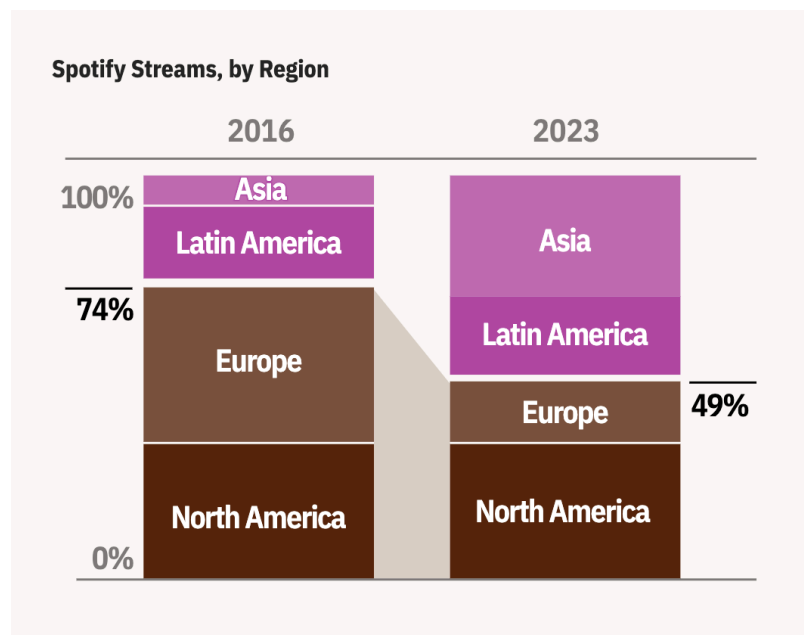


Figure 5. Spotify Streams, by Region, 2016 vs. 2023
from The Pudding

Ultimately, without the full scope of data that Spotify has access to, it is difficult to replicate the accuracy of their genre classification algorithms. We would argue that the opaqueness of Spotify’s recommendation algorithms is a good thing. If the company’s genre classification models were transparent, or any of their other recommendation algorithms for that matter, artists might be inclined to transform their approaches to music creation to increase potential for discovery and revenue. An artist might seek to optimize their music for recommendation models—to make algorithmic music—instead of maintaining their original pursuit of artistic excellence. We would argue that this scenario would be net negative for the music industry. Artists should remain focused on their commitments to artistic excellence and full creativity, and algorithms should be built and designed to support that.